

From Attention to Transformers: The Foundations of Modern Language Models

IOAI Preparation

2026



Introduction

Transformers are neural networks designed to process sequences. Their central mechanism, called **attention**, allows each token of a sequence to retrieve information from other tokens.

This problem set has two goals:

- **Attention View:** understand mathematically how self-attention works, why queries, keys and values are introduced, and how a Transformer block is built.
- **Language Modeling View:** understand how Transformers are trained to predict the next token, why causal masking is necessary, and what changes at inference time.

The objective is not to memorize an architecture diagram, but to reconstruct the main ideas from simple mathematical principles.

0) Give examples of tasks for which Transformers are used. What makes sequence data different from ordinary tabular data?

1 Attention View

1.1 From Tokens to Vectors

A tokenizer transforms a sentence into token indices

$$t_1, \dots, t_n,$$

where each token belongs to a vocabulary of size V .

A neural network cannot directly manipulate words. Instead, we associate each token with a vector using an embedding matrix

$$E \in \mathbb{R}^{V \times d}.$$

If token t_i appears at position i , its embedding is

$$x_i = E_{t_i} \in \mathbb{R}^d.$$

Stacking all token embeddings gives

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

1. Suppose the vocabulary contains $V = 50\,000$ tokens and $d = 512$. How many parameters does the embedding matrix contain?
2. If the sentence contains $n = 20$ tokens, what is the shape of X ?
3. Let $e_j \in \mathbb{R}^V$ denote the one-hot vector corresponding to token j . Show that its embedding can be written as

$$E^\top e_j.$$

4. Why might dense learned embeddings be more useful than one-hot vectors?
5. Consider the two sentences:

“dog bites man” and “man bites dog”.

Why are token embeddings alone insufficient to distinguish the role of each word?

6. A simple solution is to add a positional vector p_i to each token:

$$z_i = x_i + p_i.$$

Explain what information p_i provides.

1.2 Self-Attention

We now want each token to collect information from other tokens.

From the input matrix X , we compute three new representations:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where, for simplicity,

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}.$$

The rows of Q, K, V are called respectively **queries**, **keys**, and **values**.

For token i and token j , define the similarity score

$$s_{ij} = q_i^\top k_j.$$

1. Give the shapes of Q , K , and V .
2. Give a geometric interpretation of the dot product

$$q_i^\top k_j.$$

3. Intuitively:

- what does a query represent?
- what does a key represent?
- what does a value represent?

4. Why do we not simply use X directly for queries, keys and values?

5. For a fixed token i , we convert the scores into weights:

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^n \exp(s_{i\ell})}.$$

Show that

$$a_{ij} \geq 0 \quad \text{and} \quad \sum_{j=1}^n a_{ij} = 1.$$

6. The output corresponding to token i is

$$o_i = \sum_{j=1}^n a_{ij} v_j.$$

Explain why attention can be interpreted as a weighted average of information coming from other tokens.

7. In matrix form, attention is written

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Give the shape of $QK^\top$ and explain what its (i, j) entry represents.

Why the factor $\sqrt{d_k}$? Assume that the coordinates of $q, k \in \mathbb{R}^{d_k}$ are independent, have mean 0 and variance 1.

1. Show that

$$\mathbb{E}[q^\top k] = 0.$$

2. Show that

$$\text{Var}(q^\top k) = d_k.$$

3. Deduce that the typical magnitude of the dot product grows like $\sqrt{d_k}$.

4. Show that

$$\text{Var} \left(\frac{q^\top k}{\sqrt{d_k}} \right) = 1.$$

5. Why might very large scores make the softmax almost one-hot? Why could this make optimization harder?

6. Conclude on the role of the factor $1/\sqrt{d_k}$.

1.3 Building a Transformer Block

A Transformer does not use only one attention mechanism. It uses several attention heads in parallel.

For each head h :

$$O_h = \text{softmax} \left(\frac{Q_h K_h^\top}{\sqrt{d_k}} \right) V_h.$$

The outputs are concatenated:

$$O = \text{Concat}(O_1, \dots, O_H) W_O.$$

1. Suppose $d = 768$ and the Transformer uses $H = 12$ heads. If the dimensions are split equally, what is the dimension of each head?
2. Why might several heads be more useful than a single attention mechanism? Give examples of different relations that different heads could learn.
3. After attention, each token is passed independently through a small neural network:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

Which part of the Transformer mixes information between tokens: attention or the FFN?

4. Explain why a non-linearity σ is needed in the FFN.
5. A Transformer also uses residual connections:

$$y = x + F(x).$$

Show that

$$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}.$$

Why can the identity term help gradients propagate through deep networks?

6. A simplified Transformer block can be written as

$$X_1 = X + \text{MHA}(\text{LN}(X)),$$

$$X_2 = X_1 + \text{FFN}(\text{LN}(X_1)).$$

Identify:

- the attention operation,
 - the two residual connections,
 - the feed-forward operation,
 - the normalization.
7. Alice says: “Attention is only a weighted average, so it cannot represent complicated functions.” Bob disagrees.

Who is right? Explain why the fact that attention is a weighted average does not mean that an entire Transformer is a linear or simple model.

2 Language Modeling View

2.1 Causal Attention

A language model predicts a sequence from left to right.

For tokens

$$t_1, \dots, t_n,$$

the chain rule gives

$$p(t_1, \dots, t_n) = \prod_{i=1}^n p(t_i \mid t_1, \dots, t_{i-1}).$$

1. Derive this factorization from the chain rule of probability.
2. When predicting token t_i , why must the model not access $t_{i+1}, \dots, t_n$?
3. We define the causal mask

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases}$$

Explain why

$$\text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right)$$

prevents attention to future tokens.

4. Draw the mask for a sequence of length 5.
5. What would happen during training if we forgot this causal mask?

2.2 Training: Maximum Likelihood

At each position i , the Transformer produces logits

$$z_i \in \mathbb{R}^V.$$

Applying softmax gives

$$p_\theta(t = j \mid t_{<i}) = \frac{\exp(z_{i,j})}{\sum_{k=1}^V \exp(z_{i,k})}.$$

For a training sentence

$$t_1, \dots, t_n,$$

the likelihood is

$$L(\theta) = \prod_{i=1}^n p_\theta(t_i \mid t_{<i}).$$

1. Write the log-likelihood of the training sequence.
2. Show that maximizing the likelihood is equivalent to minimizing

$$-\sum_{i=1}^n \log p_\theta(t_i \mid t_{<i}).$$

3. Explain why this is the usual cross-entropy loss.
4. Suppose the correct token has predicted probability 0.9. Compute its contribution to the loss.
5. Do the same if the predicted probability is 0.01. What does this tell you about confident wrong predictions?
6. Consider the sentence

“The cat sits on the mat”.

What is the prediction target after the prefix

“The cat sits”?

7. During training, all true previous tokens are known. Explain why all positions can be processed in parallel despite the autoregressive factorization.

2.3 Inference and the KV Cache

At inference time, the true next token is unknown. The model generates one token, adds it to the sequence, and repeats.

1. Describe how a language model generates three new tokens starting from a prompt.
2. What is the difference between:
 - greedy decoding,
 - sampling from the predicted distribution?
3. During generation, suppose the model has already processed

$$t_1, \dots, t_n.$$

When generating t_{n+1} , do the old key vectors and value vectors of tokens $1, \dots, n$ change?

4. Explain why they can therefore be stored and reused. This is called the **KV cache**.
5. Why must we still compute a new query for the newest token?
6. What happens to the size of the KV cache as the sequence length increases?
7. Standard attention uses the matrix

$$QK^\top \in \mathbb{R}^{n \times n}.$$

Explain why standard attention has quadratic complexity in sequence length.

8. Alice proposes using only nearby tokens in attention. Bob proposes keeping exact attention but computing it more efficiently using FlashAttention.

Explain the conceptual difference between these two ideas.

2.4 Final Challenge: Build a Small Attention Layer

We now want to implement a causal self-attention layer using NumPy.

You are given

$$X \in \mathbb{R}^{n \times d}$$

and weight matrices

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}.$$

1. Compute

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

2. Compute the score matrix

$$S = \frac{QK^\top}{\sqrt{d_k}}.$$

3. Construct a causal mask and add it to S .

4. Implement row-wise softmax.

For numerical stability, use:

$$\text{softmax}(z)_j = \frac{\exp(z_j - \max_k z_k)}{\sum_\ell \exp(z_\ell - \max_k z_k)}.$$

5. Compute the attention matrix A and output

$$O = AV.$$

6. Check numerically that each row of A sums to 1.

7. Check that entries corresponding to future tokens are zero.

8. Change a future token x_j with $j > i$. Verify that the output of position i does not change.

9. **Bonus:** Extend the implementation to multi-head attention.

10. **Bonus:** Simulate autoregressive generation and implement a simple KV cache.

2.5 Conclusion

Make a report of the main takeaways for these problems. You don't need to make a good looking report, just make sure you write down what you understood somewhere.

In particular, make sure you can explain:

- what queries, keys and values represent,
- what the attention matrix contains,
- why we divide by $\sqrt{d_k}$,
- why Transformers need positional information,
- the role of multi-head attention, FFNs and residual connections,

- why causal masking is necessary,
- why next-token training corresponds to maximum likelihood,
- the difference between training and inference,
- why the KV cache accelerates autoregressive generation.